In [ ]:
```
#Name : Sharvil Bharatesh Magdum
#PRN : 2425000909
#Roll No : A33
#Batch : A - 2
#Assignment No : 10 (Machine Learning -> K-Means Clustering)
```

In [8]:
```
#1. Import basic libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [11…
```
#2. Import the 'housing.csv' dataset
data = pd.DataFrame(pd.read_excel("housing.xlsx"))
data
```

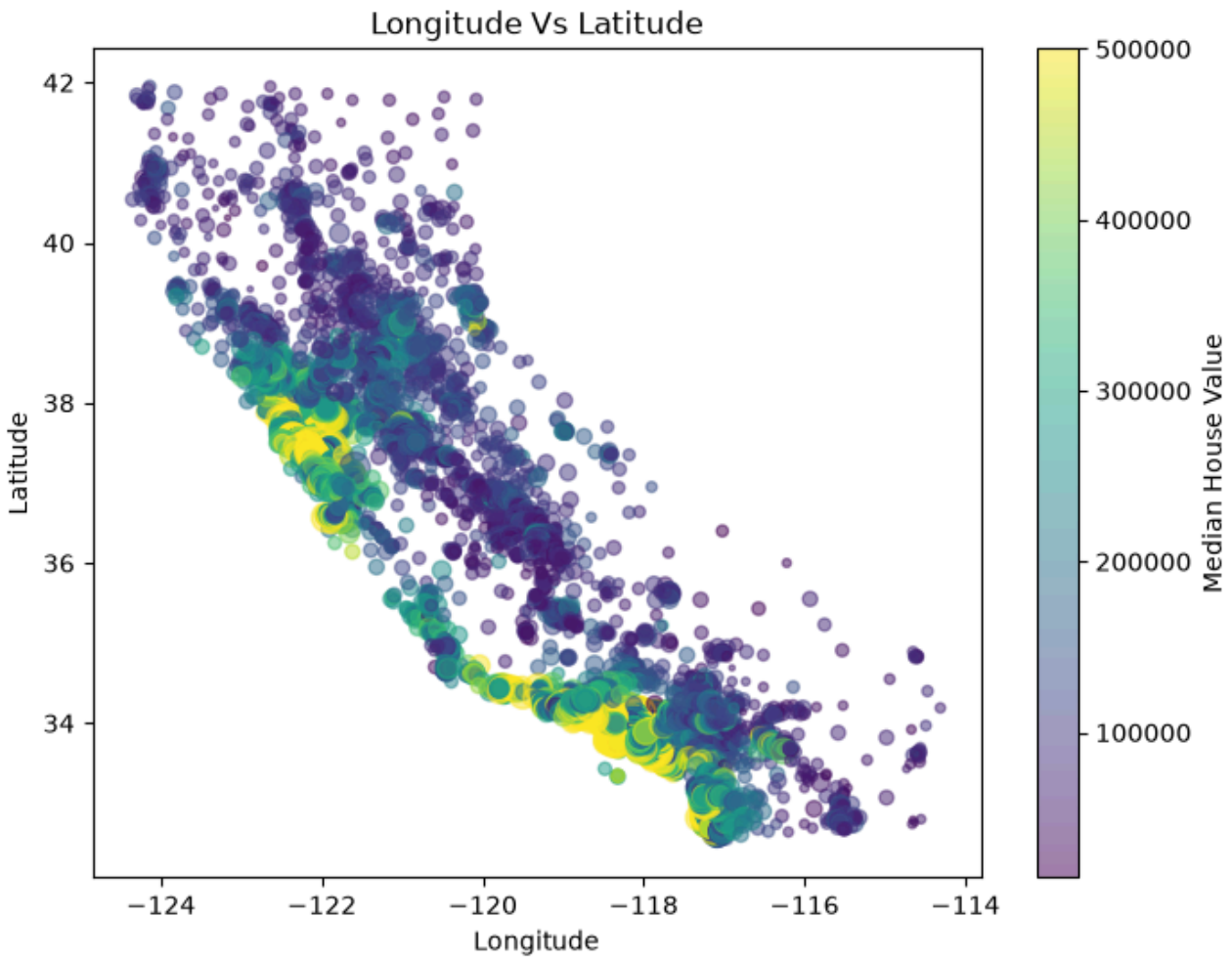
Out[11]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	h
0	-122.23	37.88	41	880	129.0	322	
1	-122.22	37.86	21	7099	1106.0	2401	
2	-122.24	37.85	52	1467	190.0	496	
3	-122.25	37.85	52	1274	235.0	558	
4	-122.25	37.85	52	1627	280.0	565	
…	…	…	…	…	…	…	
20635	-121.09	39.48	25	1665	374.0	845	
20636	-121.21	39.49	18	697	150.0	356	
20637	-121.22	39.43	17	2254	485.0	1007	
20638	-121.32	39.43	18	1860	409.0	741	
20639	-121.24	39.37	16	2785	616.0	1387	

20640 rows × 10 columns

In [13…
```
#3. Select any three features from the above dataset
x = data[['longitude','latitude','median_income']]
y = data['median_house_value']
```

In [16…
```
#4. Visualize the original data using a scatter plot
plt.figure(figsize=(8,6))
scatter = plt.scatter(x['longitude'],x['latitude'],c = y,cmap = 'viridis'
plt.colorbar(scatter,label='Median House Value')
plt.title('Longitude Vs Latitude')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```



In [19…

```
#5. Figure out the x and y variables
print("Features for x:\n",data[["longitude",'latitude','median_income']])
print()
print("Features for y:\n",data['median_house_value'])
```

```
Features for x:
        longitude  latitude  median_income
0         -122.23     37.88         8.3252
1         -122.22     37.86         8.3014
2         -122.24     37.85         7.2574
3         -122.25     37.85         5.6431
4         -122.25     37.85         3.8462
...           ...       ...            ...
20635     -121.09     39.48         1.5603
20636     -121.21     39.49         2.5568
20637     -121.22     39.43         1.7000
20638     -121.32     39.43         1.8672
20639     -121.24     39.37         2.3886

[20640 rows x 3 columns]

Features for y:
 0         452600
1         358500
2         352100
3         341300
4         342200
          ...
20635      78100
20636      77100
20637      92300
20638      84700
20639      89400
Name: median_house_value, Length: 20640, dtype: int64
```

In [21…
```
#6. Perform the train and test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2,rand
```

In [26…
```
#7. Print the shape of the train and test variables
print("x_train size:",x_train.shape)
print("x_test size:",x_test.shape)
print("y_train size:",y_train.shape)
print("y_test size:",y_test.shape)
```

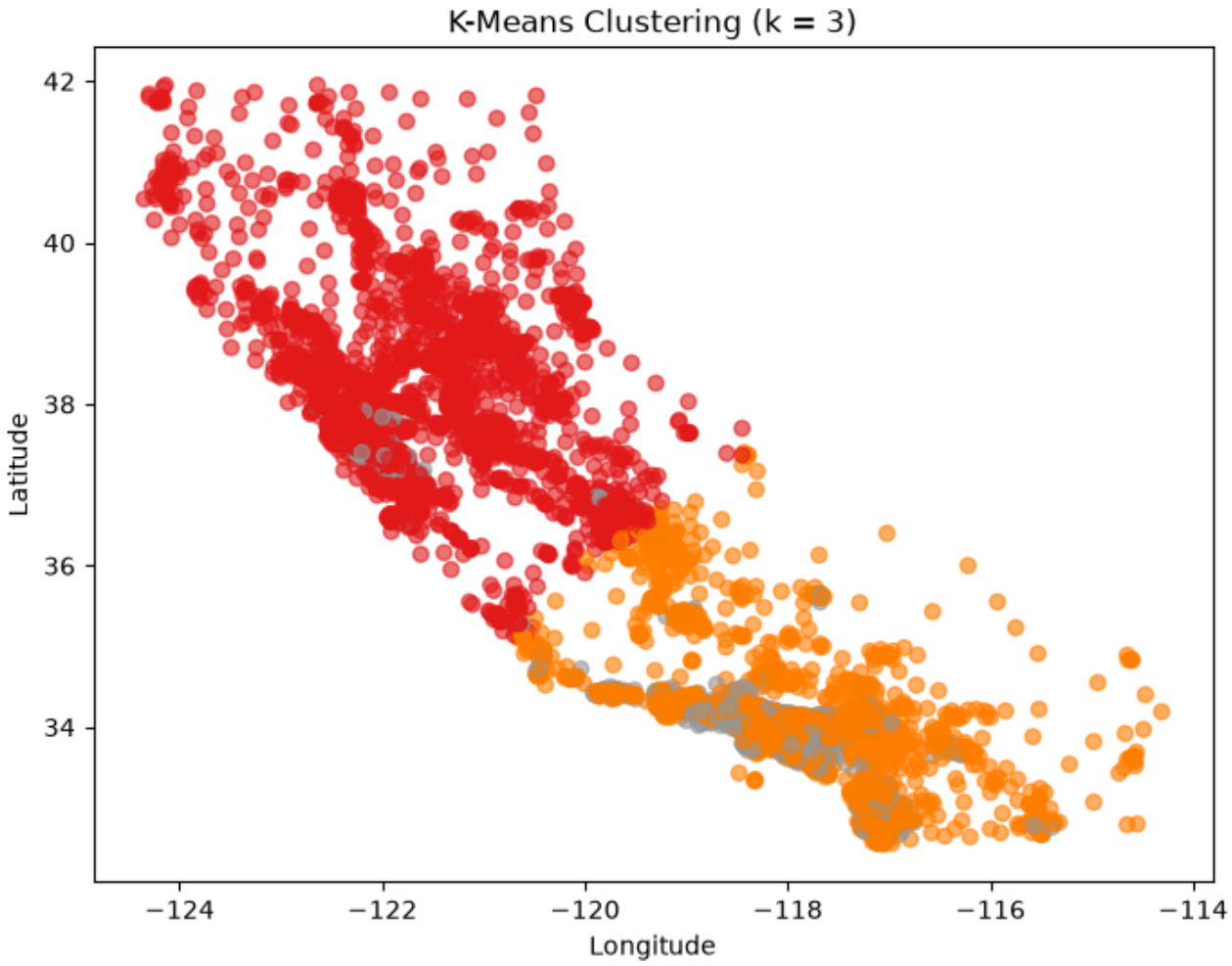
```
x_train size: (16512, 3)
x_test size: (4128, 3)
y_train size: (16512,)
y_test size: (4128,)
```

In [30…
```
#8. Perform the data normalization on the input x
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.fit_transform(x_test)
```

In [52…
```
#9. Train the k-means model with value of k=3 taken arbitrarily
from sklearn.cluster import KMeans
k_means = KMeans(n_clusters = 3,random_state = 42,n_init = 10)
```

```
train_clusters = k_means.fit_predict(x_train_scaled)
```

In [54…
```
# #10. Visualize the trained model using scatter plots with k=3 to displa
plt.figure(figsize=(8,6))
plt.scatter(x_train['longitude'],x_train['latitude'],c = train_clusters,c
plt.title("K-Means Clustering (k = 3)")
plt.xlabel("Longitude")
plt.ylabel('Latitude')
plt.show()
```

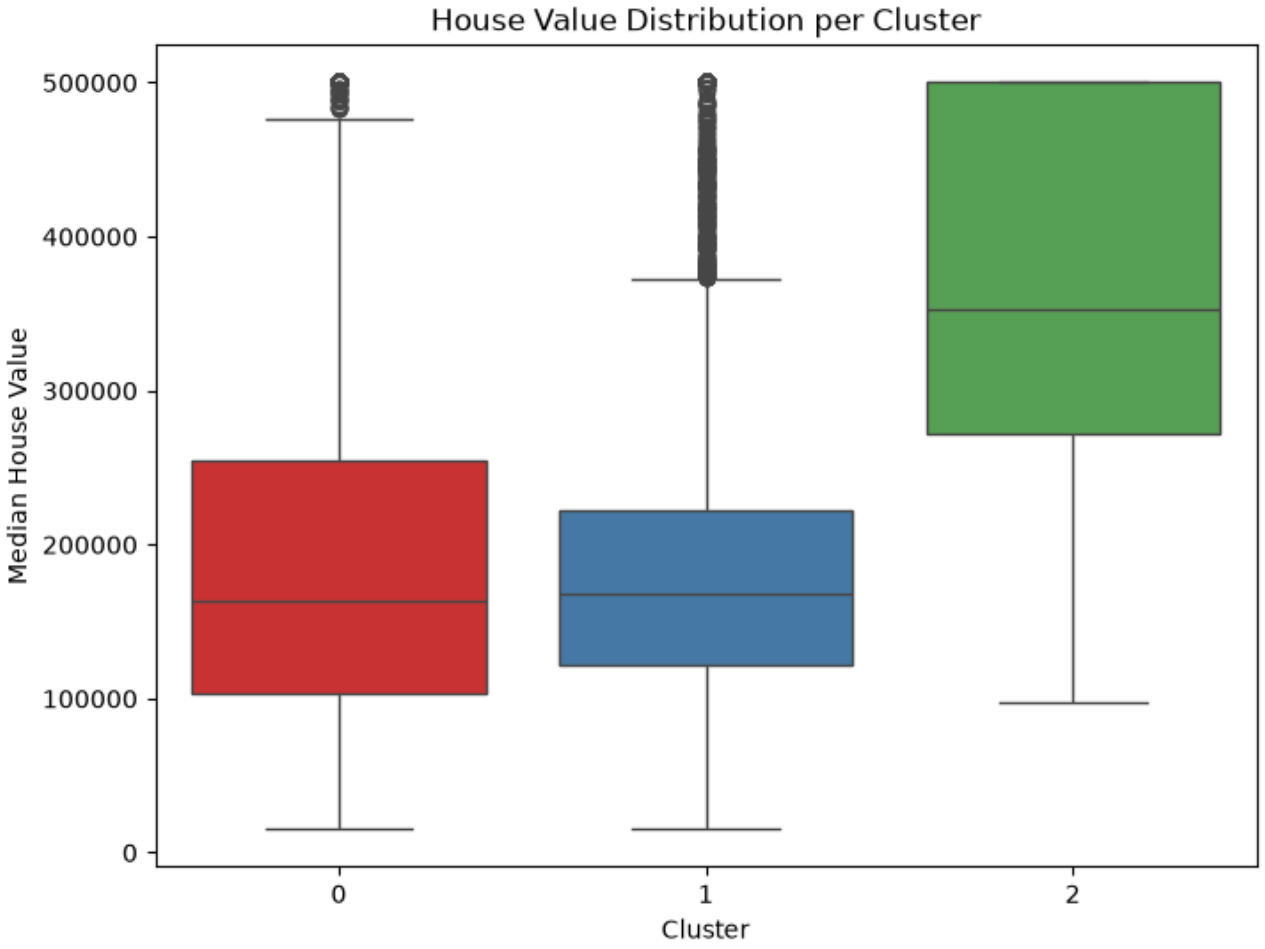


In [77…
```
#11. Visualize the trained model using boxplots with k=3 to display house
plt.figure(figsize=(8,6))
sns.boxplot(x = train_clusters,y = y_train,palette = 'Set1')
plt.title("House Value Distribution per Cluster")
plt.xlabel('Cluster')
plt.ylabel('Median House Value')
plt.show()
```

```
C:\Users\Atharv\AppData\Local\Temp\ipykernel_6016\426186003.py:3: FutureW
arning:

Passing `palette` without assigning `hue` is deprecated and will be remov
ed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo
r the same effect.

  sns.boxplot(x = train_clusters,y = y_train,palette = 'Set1')
```



In [61…
```
#12. Evaluate the performance of the algorithm using Silhouette score for
from sklearn.metrics import silhouette_score
sil_score = silhouette_score(x_train_scaled,train_clusters)
```

In [64…
```
#13. Using Elbow method find out the best value of k for the given datase
# Print each k and the silhouette score
k_values = range(2,11)
inertias = []
silhouette_scores = []

for k in k_values:
    k_means = KMeans(n_clusters = k,random_state = 42,n_init = 10)
    clusters = k_means.fit_predict(x_train_scaled)

    inertia = k_means.inertia_
```

```
    sil_score = silhouette_score(x_train_scaled,clusters)

    inertias.append(inertia)
    silhouette_scores.append(sil_score)

    print(f"k = {k} | Inertia: {inertia:.2f} | Silhouetter Score: {sil_sc
```

```
k = 2 | Inertia: 21504.22 | Silhouetter Score: 0.5473
k = 3 | Inertia: 15247.45 | Silhouetter Score: 0.5154
k = 4 | Inertia: 11639.22 | Silhouetter Score: 0.4328
k = 5 | Inertia: 9509.21 | Silhouetter Score: 0.3869
k = 6 | Inertia: 8069.19 | Silhouetter Score: 0.3987
k = 7 | Inertia: 7020.68 | Silhouetter Score: 0.3718
k = 8 | Inertia: 6201.63 | Silhouetter Score: 0.3552
k = 9 | Inertia: 5630.97 | Silhouetter Score: 0.3581
k = 10 | Inertia: 5117.29 | Silhouetter Score: 0.3410
```

In [70…

```
#14. Print the best value of k and the corresponding best silhouette scor
best_k = k_values[np.argmax(silhouette_scores)]
best_sil_score = max(silhouette_scores)
print(f"The best value of K is {best_k} with Silhouette Score {round(best
```

```
The best value of K is 2 with Silhouette Score 0.5473
```

In [71…

```
#15. Plot the Elbow plot vs Silhouette Score
fig , ax1 = plt.subplots(figsize=(10,6))
color = 'tab:blue'
ax1.set_xlabel("Number of clusters(k)")
ax1.set_ylabel("Inertia (Elbow Method)",color = color)
line1 = ax1.plot(k_values,inertias,marker = 'o',color = color,label = "In
ax1.tick_params(axis = 'y',labelcolor = color)

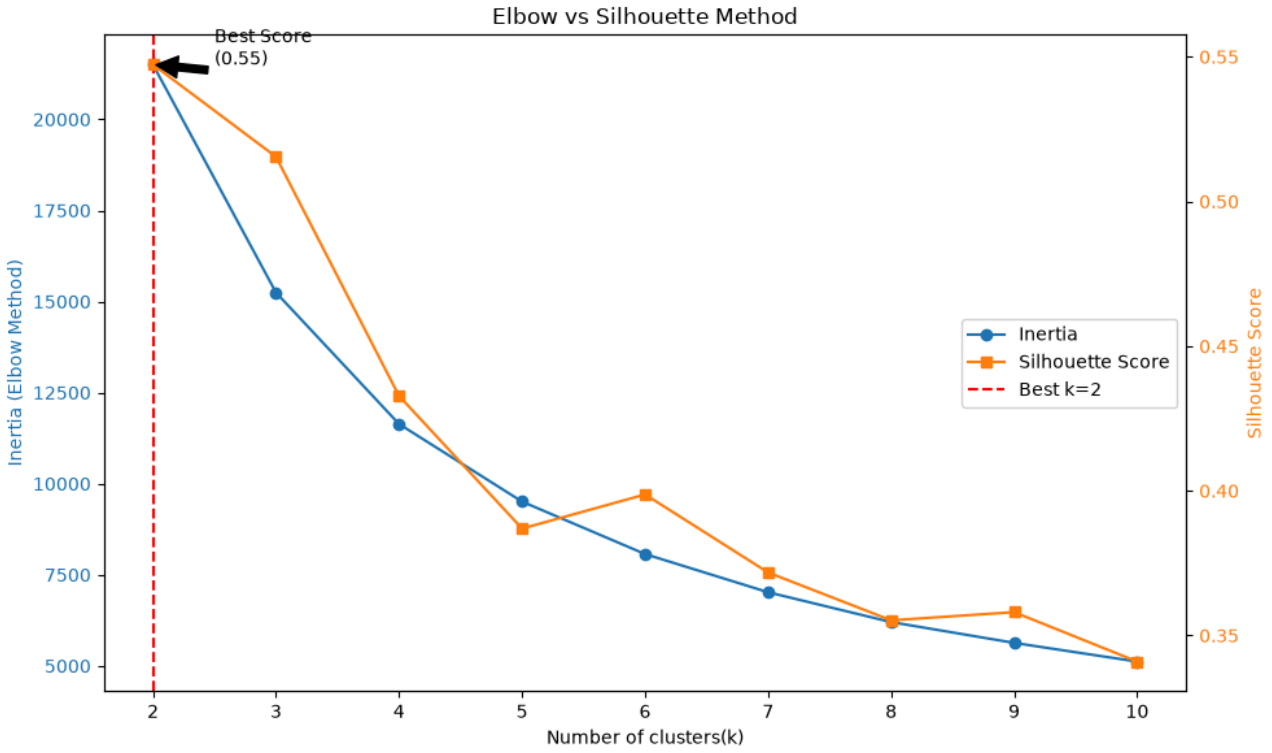
ax2 = ax1.twinx()
color = 'tab:orange'
ax2.set_ylabel('Silhouette Score',color = color)
line2 = ax2.plot(k_values,silhouette_scores,marker = 's',color = color,la
ax2.tick_params(axis = 'y',labelcolor = color)

# Plot a vertical line at best value of k.
ax1.axvline(x = best_k,color = 'red',linestyle = '--',label = f"Best K =

ax2.annotate(f'Best Score\n({best_sil_score:.2f})',
             xy=(best_k, best_sil_score), xytext=(best_k+0.5, best_sil_sc
             arrowprops=dict(facecolor='black', shrink=0.05))

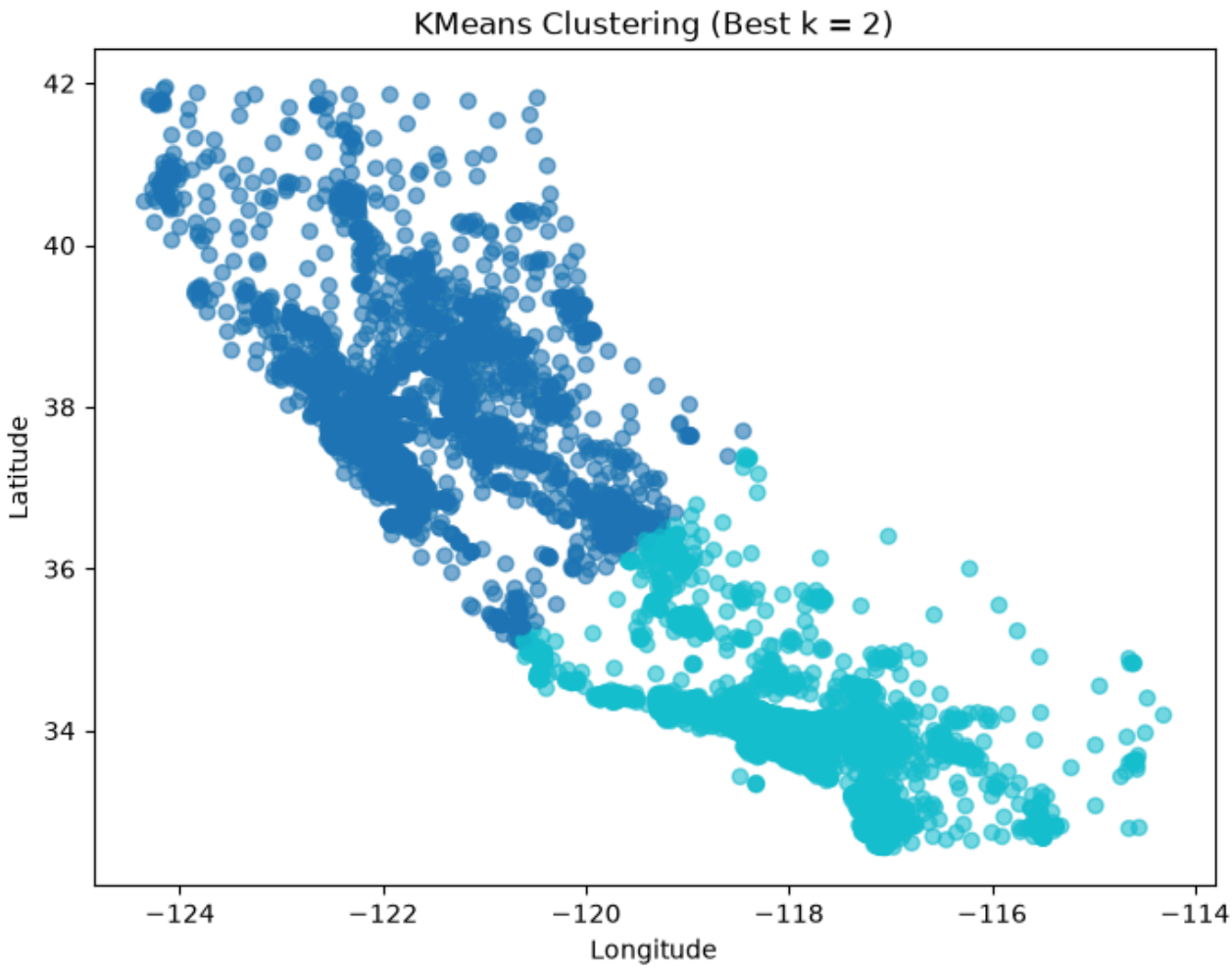
# Plot title as "KMeans Clustering: Elbow vs Silhouette Method"
# Annotate best silhouette score point
lines = line1 + line2 + [plt.Line2D([0], [0], color='red', linestyle='--'
labels = [l.get_label() if hasattr(l, 'get_label') else f'Best k={best_k}

# Display Legend combining both axes
ax1.legend(lines, ['Inertia', 'Silhouette Score', f'Best k={best_k}'], lo
plt.title("Elbow vs Silhouette Method")
fig.tight_layout()
plt.show()
```



In [72…
```
#16. Retrain the model using the best value of k
kmeans_best = KMeans(n_clusters = best_k,random_state = 42,n_init = 10)
train_clusters_best = kmeans_best.fit_predict(x_train_scaled)
```

In [73…
```
#17. Visualize the trained model using scatter plots with best value k to
plt.figure(figsize=(8,6))
plt.scatter(x_train['longitude'],x_train['latitude'],c = train_clusters_b
plt.title(f'KMeans Clustering (Best k = {best_k})')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```



In [76…

```
#18. Visualize the trained model using box plots with best value k to dis
plt.figure(figsize=(8, 6))
sns.boxplot(x=train_clusters_best, y=y_train, palette='tab10')

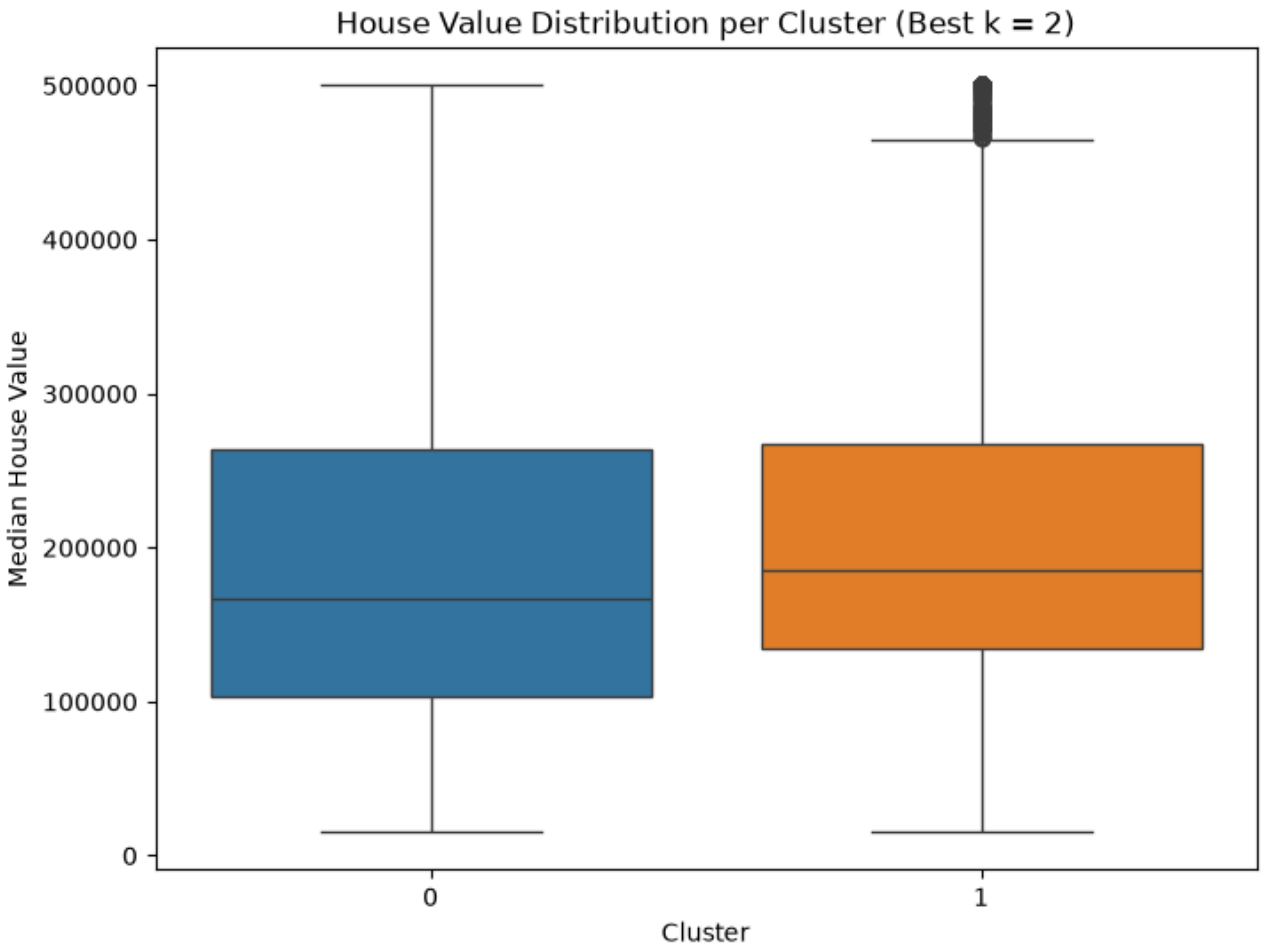
# Use an f-string (and best_k if that is what you named your variable)
plt.title(f"House Value Distribution per Cluster (Best k = {best_k})")

plt.xlabel('Cluster')
plt.ylabel('Median House Value')
plt.show()
```

```
C:\Users\Atharv\AppData\Local\Temp\ipykernel_6016\3137062814.py:3: Future
Warning:

Passing `palette` without assigning `hue` is deprecated and will be remov
ed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` fo
r the same effect.

  sns.boxplot(x=train_clusters_best, y=y_train, palette='tab10')
```


House Value Distribution per Cluster (Best k = 2)
Median House Value
500000
400000
300000
200000
100000
0
0
1
Cluster